

Ch #1

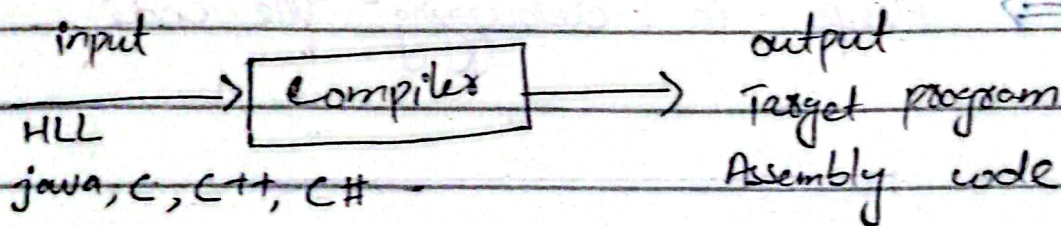
Q. What is Compiler?

Compiler:-

"A Compiler is a computer program that translates a program written in one language (source code) into an equivalent program (target language) in another."

⇒ It is a system software that converts source code written in a programming language into machine code that is understood by computer.

⇒ It converts high level language code into low level language code that is understood by processors. ⇐



Q. What is Compiler Construction?

Compiler Construction:-

"The process of constructing a compiler is called compiler construction."

Q. Why we construct compiler?

⇒ Without (compilers) are not able to understand the source code written in high level language and this our required tasks will not get executed.

Q. List certain features of compiler?

Features:

- ⇒ Preserve the correct meaning of the code.
- ⇒ Recognize legal and illegal program constructs.
- ⇒ Good error reporting
- ⇒ Helps in debugging the code.

⇒ The compiling process contains essential translation operation and error detection.

Q. Define the different types of compiler?

These are different types of compiler but based on passes there are two types:

* Based on Passes:

Single Pass

⇒ It convert directly source code into machine code.

⇒ It compiles the whole program in one pass.

Pascal

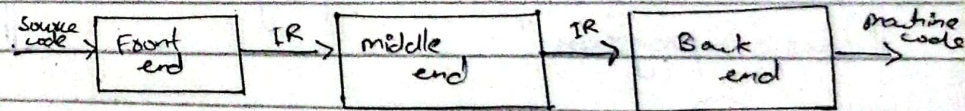
Multi-Pass

⇒ It compiles source code of a program several times.

⇒ It compile the source code of a program time.

Java

Source code → Compiles → Target code



Q. Elaborate Single Pass Compilers:
 ⇒ when all phases of compiler design in a single module, then it is called a single pass compiler.

Q. Two Pass Compiler:
 - Program is translated twice:
 • one from front end
 • one from the backend
 Source code → Front end → IR → Back end → Target code
 - allows multiple front end

Incremental Compiler:

Incremental Compiler is a compiler which performs the recompilation of only modified source code rather than compiling the whole source program.

Programmer writes the code → Compiler translates it to machine code

- it tracks dependencies b/w op and the source program.
 - it produces the same result as full recompilation.
 - recompiles less code than full recompilation.
2) Cross platform Compilers:
 ⇒ They produce an executable machine code for a platform this platform is not the one on which compiler is running.

(3) Bootstrap Compiler:-
 - Bootstrapping is used to produce a self-hosting compiler that can compile its own source code.
 ⇒ These compilers are written in a programming language that they have to compile. Used to compile the compiler and the compiler compiles the source code.
4) Load and go compilers:
 - Code produced from such compiler is either ready for execution or in need of further transformation by a linker and loader into absolute code.

ex: WATFOR Compiler

5) Just-in-time Compilers:

ex
 - JIT Compiler time optimization of java program
 - CLR for C#

Q. Difference between Compiler & interpreter?

Compiler

- ⇒ Translate and execute whole program
- ⇒ Intermediate code generated
- ⇒ more efficient / fast
- ⇒ Non-scripting languages
- ⇒ Reports all errors in program
- ⇒ Program cannot execute if any error
- e.g C compiler

Interpreter

- ⇒ Translate and execute line by line
- ⇒ No generated code
- ⇒ Slow
- ⇒ Scripting language
- ⇒ Stops at first error of program
- ⇒ Program can partially execute if errors
- e.g Basic

Features of static Compiler

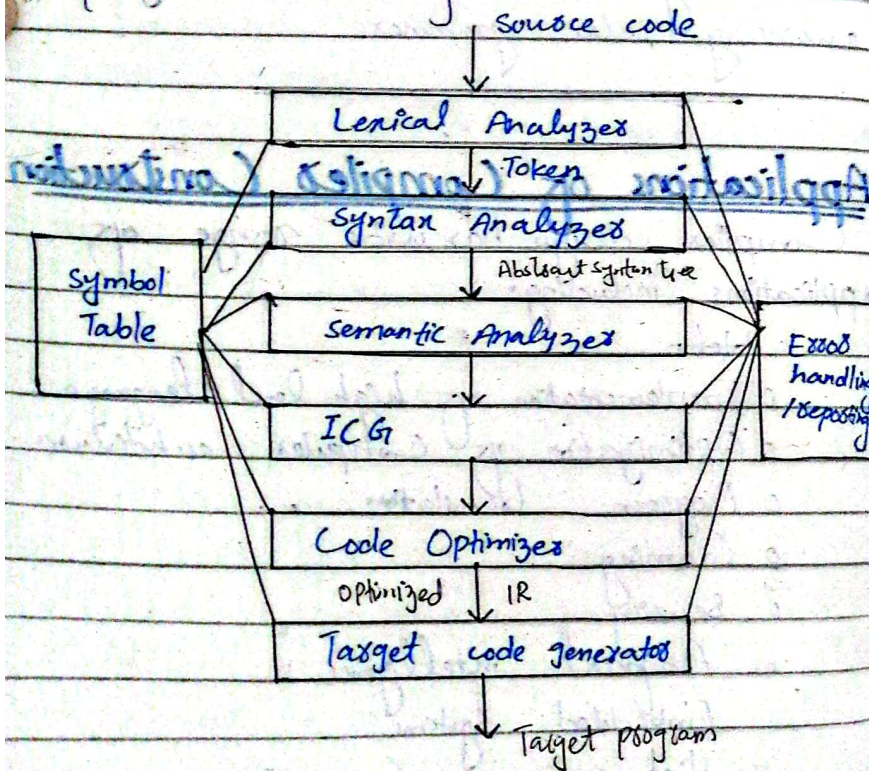
- ⇒ The features as follows:
- ⇒ Compilation Speed
- ⇒ The correctness of machine code.
- ⇒ The meaning of code should not change.
- ⇒ Speed of machine code.
- ⇒ Good error detection
- ⇒ Checking the code correctly according to programmer.

Applications of Compiler Constructor

- ⇒ Compiler design has wide range of applications including:
 - o Implementation of high-level language
 - o Optimization of computer architecture
 - o Program translations
 - o Naming
 - o Security
 - o Artificial intelligence
 - o Embedded systems
 - o High performance computing

Phases / Structure of Compiler

- ⇒ The compilation process takes place in several phases.
- ⇒ For each step, the output of one step acts as the input for the next step.
- ⇒ The phases / structure of the compilation process are as follows:



1) Lexical analyzer:-

- ⇒ It takes the high-level language source code as the input.
- ⇒ It scans the characters of source code from left to right. Hence, also called scanner.
- ⇒ It groups the characters into lexemes. Also known as Tokenizer. EX: $y = 2 * x$
 $\text{y} \quad \text{=} \quad \text{2} \quad \text{*} \quad \text{x}$
 Lexemes: are group of characters which has some meaning.
- ⇒ Each lexeme corresponds to from a token.

- ⇒ It removes white spaces and comments and blank spaces.
- ⇒ It checks for lexical errors. I/P: Stream of characters
O/P: Stream of tokens

(2) Syntax analyzer:-

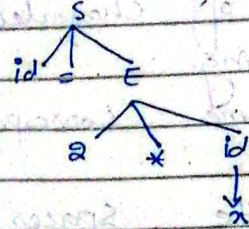
- ⇒ Parser is one of the other name of syntax analyzer.
- ⇒ The output of lexical analyzer is its input.
- ⇒ It checks for syntax errors in the source code.

⇒ It does this by constructing a parse tree of all the tokens.

⇒ For the syntax to be correct, the parse tree should be according to the rules of source code grammar.

⇒ The grammar for such codes is Context-free grammar.

$y = x * x$



(3) Semantic Analyzer:

⇒ It verifies the parse tree of the syntax analyzer.

⇒ It checks the validity of the code in terms of programming language. Like, Compatibility of data types, declaration, and initialization of variables, etc.

⇒ It checks logical errors.

⇒ It also performs reachability analysis. Furthermore, we also call this tree an annotated parse tree.

⇒ It also performs flow checking, type checking.

⇒ It does not check a number divided by zero because that is checked at run time.

⇒ It just does compile time checking.

(4) ICG / Intermediate code generator

⇒ It generates an intermediate code.

⇒ This code is neither in high-level language or low level language. It is an intermediate form.

⇒ It is converted to machine language but, the last two phases are platform dependent.

⇒ The intermediate code is same for all compilers. Further, we generate the machine code platform dependent.

⇒ e.g. Three address code.

(5) Code Optimizer:

- ⇒ It optimizes the intermediate code
- ⇒ Its function is to convert the code so that it executes faster (CPU memory).
- Using fewer resources
- ⇒ It removes any useless lines of code and rearranges the code.
- ⇒ The meaning of the source code remains the same.

(6) Target Code Generator:

- ⇒ Finally, it converts the optimized intermediate code into the machine code.
- ⇒ This is the final stage of compilation.
- ⇒ The machine code which is produced is relocatable.

Note: All these phases are divided into two sections:

- front-end
- back-end

Short terms

Q. What is Symbol table?

⇒ It is a data structure used by language translators for storing information about identifiers (variables, names, functions etc) and their attributes (address space etc). It is a type of memory.

Eg: Public int x;

⇒ it helps in different compilation tasks.

Name	Type	Location	Scope
<u>name</u>			<u>scope</u>

Q. What is error reporting or error handling?

⇒ When it detects errors.

⇒ (1) Report it to the user.

⇒ (2) Make some recovery strategies and implement it to handle exceptions.

- e.g.
- Panic mode recovery
 - Phase level recovery

Types of errors:

<u>Lexical errors</u>	<u>Semantical errors</u>	<u>Syntactical errors</u>

misspelling of identifiers or keywords or operators.	incomplete value assign name	Missing semicolon or colon or unbalanced parenthesis.
	Type mismatch blw operators & operands	

Q. Difference between tokens & Lexeme?

Token

⇒ Valid sequence of characters in a language are called tokens.

e.g. Examples of Tokens in language:

Constants

Identifiers (Y, x, class, z)

Operators (, {, +, ++)

Punctuations ; , ' "

Keywords (int, if, while)

Some tokens can include single character

Lexeme

⇒ Sequence of characters matched by pattern forming tokens. OR specific instance of token class is called Lexeme.

for example: "int x = 5;"

the lexemes would be "int", "x", "=", "5", and ";"

misspel took

e.g. { if, ;, C, 10, ++, int, x, public, variable, {, while, ,, -, etc. }

Q. What are patterns?

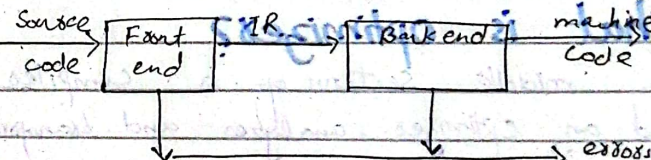
⇒ Set of rules that define a token OR

⇒ Set of rules for formation of tokens from input characters

e.g. Identifiers none must start with underscore or alphabets.

Summary:

⇒ To sum up these are two phases of compiler structure: Front end and back end



Responsibilities:

- Recognize legal & illegal programs
- Reports errors in a useful way
- Produce IR & Preliminary storage map
- Shape the code for back end IR

Front end: must front end compiler can be automated

The front end checks whether the programs is correctly written in terms of the programming language syntax and semantics.

Back end:

- 1) Logical analysis
- 2) Preprocessing
- 3) syntax analysis
- 4) semantic analysis

The back end is responsible for translating the source into assembly code.

- Analysis
- Optimization
- Code generation

* Define instruction set?

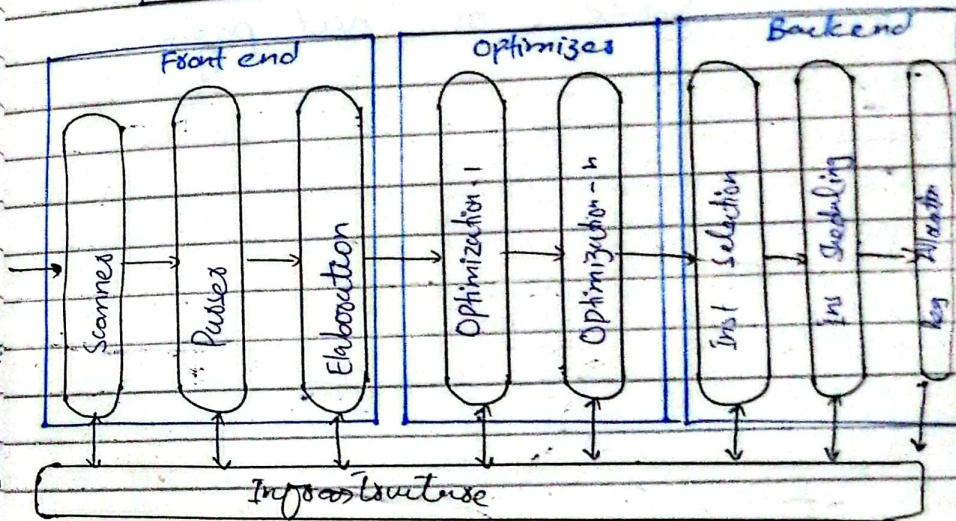
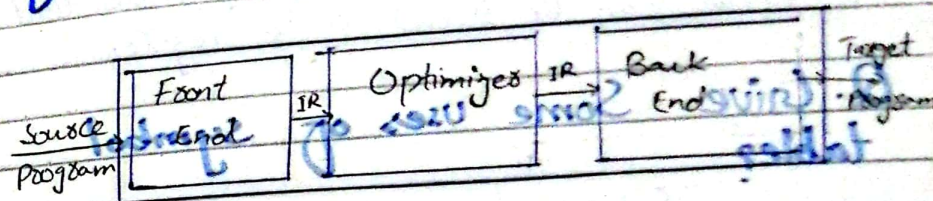
The set operations supported by processor; the overall design of an instruction set is often called an instruction set architecture or ISA.

* What is optimizer?

The middle section of a compiler, called an optimizer, analyzes and transforms the IR to improve it.

The optimizer is an IR-to-IR transformer that tries to improve the IR program in some way.

Q. Draw diagram of structure of a typical compiler?



Q. What is virtual machine?

A virtual machine is a simulator for some processor. It is an interpreter for that machine's instruction set.

Q. What is virtual register?

⇒ It is a symbolic register name that the compiler uses to indicate that a value can be stored in a register.

Q. Give Some uses of symbol table?

Solved on next page

"Content in which Compiler typically Operates."

Cousines of Compiles // Language Processing System:

(1) Preprocessor:

⇒ The preprocessor accepts source code as input and is responsible for:

- Removing Comments.
- Process all preprocessor directives denoted by #.

- The dtp is said to be preprocessed form of the pp data.

(2) Compiler:

◦ It takes pure high level language as input and convert it into assembly code.

(3) Assembler:

◦ It takes assembly language as input and convert it into relocatable machine code.

↓
whose location should be changed.

(4) Linking & Loading:

It has four functions.

(1) Allocation:

Get the memory positions from OS and stores the object data.

Relocatable

⇒ It maps the relative addresses to the physical addresses and relocates the object code.

⇒ Relocatable code is a software whose execution address can be changed.

Linker:

⇒ it is a tool used to link all parts of a program together for execution into a single executable file.

Loader:

⇒ Loader takes executable file as input and load this file into the memory and starts its execution.

